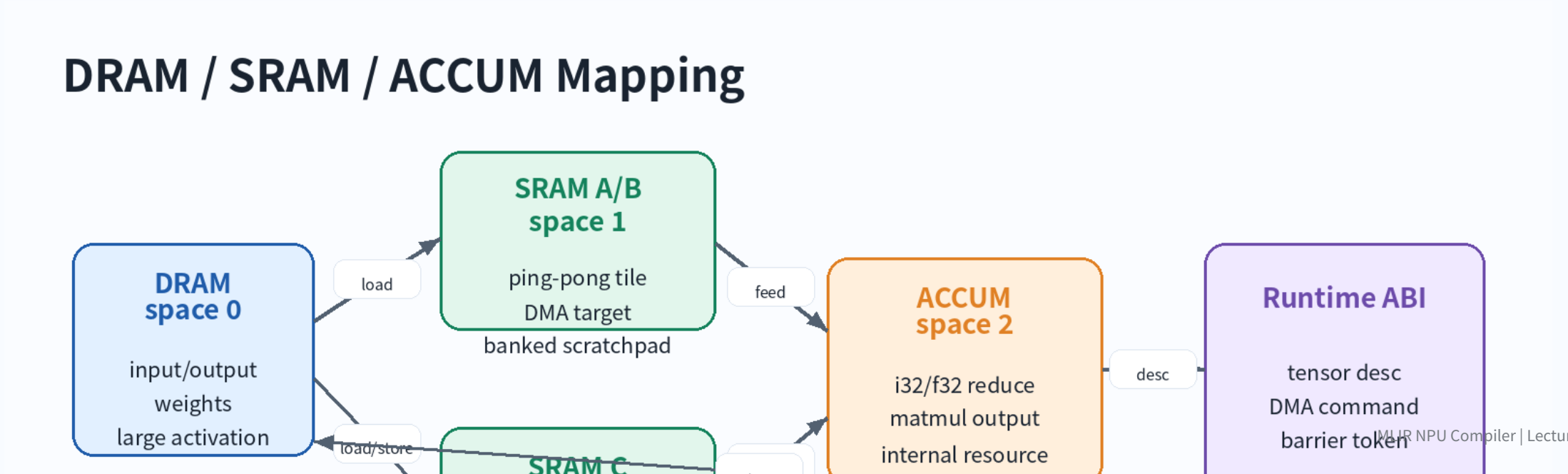
Lecture 13

MemRef, Layout, Memory Space

DRAM/SRAM/Accumulator mapping for NPU Compiler



오늘의 위치

12 강 : tensor -> memref bufferization

13 강 : memref layout and memory-space contract

14 강 : vector dialect and tensorization

NPU compiler 에서는 13 강이 memory architecture boundary 다 .

Learning Objectives

MemRef type 을 shape/type/layout/memory-space 로 분해한다 .

strided layout 과 affine map 을 NPU address generation 으로 해석한다 .

DRAM/SRAM/ACCUM memory space 를 target ABI contract 로 설계한다 .

MatMul tile buffer 의 placement, footprint, descriptor 를 작성한다 .

layout/memory-space verifier 와 FileCheck test 를 설계한다 .

왜 중요한가 ?

NPU 성능은 data movement 에 먼저 막힌다 .

layout 은 DMA burst, vector load, bank conflict 를 결정한다 .

memory space 는 allocation pool, lifetime, command buffer ABI 를 결정한다 .

잘못된 memref lowering 은 extra copy 또는 scalarized access 로 이어진다 .

Tensor vs MemRef

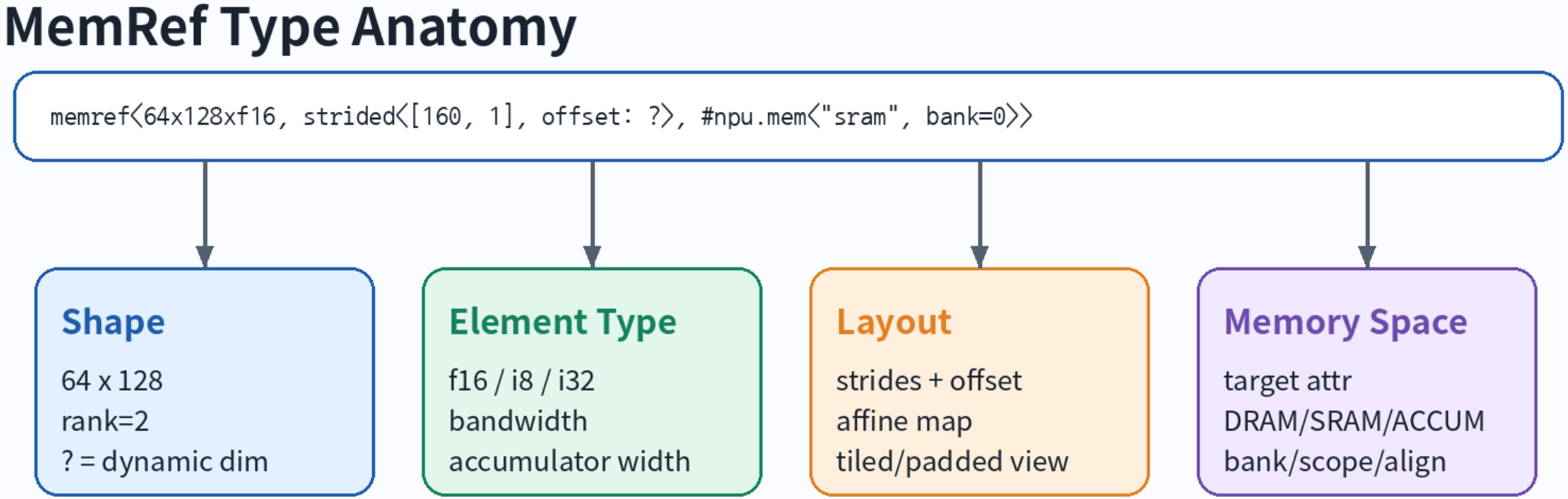
Tensor: immutable value semantics, high-level transform 에 적합

MemRef: storage semantics, allocation/view/lifetime 명시

Bufferization 이후에는 placement decision 이 visible 해야 함

NPU backend 는 memref type 을 kernel ABI 로 읽는다 .

MemRef Type Anatomy



NPU compiler contract: layout and memory space decide address generation, DMA legality, SRAM footprint, bank conflicts, and runtime ABI fields.

MemRef Type Syntax

```
memref<64x128xf16>
memref<64x128xf16, strided<[160, 1], offset: 0>>
memref<?x?xf16, strided<[?, ?], offset: ?>>
memref<64x128xf16, strided<[160,1], offset:0>, 1>
memref<64x128xf16, strided<[160,1], offset:0>, #npu.mem<"sram", bank=0>>
```

MemRef 는 pointer 가 아니다

base pointer 외에도 offset, sizes, strides 가 필요하다 .

dynamic shape/stride 는 runtime descriptor field 로 남는다 .

layout 이 identity 가 아니면 plain C pointer ABI 로 충분하지 않다 .

NPU descriptor 설계는 memref lowering 의 일부다 .

Layout 이 의미하는 것

logical index -> physical storage metadata mapping

strided form: offset + strides

affine map: tiled/padded/swizzled layout 표현 가능

NPU address generator 와 DMA descriptor 의 입력

Layout Map Examples

Layout Map and Strided Metadata

Identity row-major

```
memref<4x8xf16>
strides = [8, 1]
offset = 0
```

addr = base + (i*8+j)*2B

Subview tile

```
memref<2x4xf16,
strided<[16,2], offset:9>>
```

view metadata; no copy

Tiled layout

```
#tile = affine_map<(m,n)->
(m div 16, n div 32,
m mod 16, n mod 32)>
```

outer tile + inner tile coordinates

Banked SRAM

```
stride padded to 64B
bank=(col/8) mod 4
```

reduce bank conflicts

Strided Address Formula

```
linear_index = offset + sum_i(index_i * stride_i)
byte_address = base + linear_index * sizeof(element)

Example:
memref<64x128xf16, strided<[160, 1], offset: 0>, 1>
addr(i,j) = base + (i*160 + j) * 2
```

Subview = tile view

memref.subview 는 reduced-size view 를 만든다 .

offsets/sizes/strides 를 바꾸지만 data copy 를 의미하지 않는다 .

tile view 를 먼저 만든 뒤 DMA lowering 이 copy command 를 만든다 .

edge tile 은 dynamic sizes 로 표현될 수 있다 .

Metadata operations

memref.transpose: strided metadata permutation

extract_strided_metadata: base/offset/sizes/strides 추출

reinterpret_cast: descriptor 재구성

runtime descriptor lowering 에서 유용한 bridge op

Memory Space 관점

MLIR memref memory space 는 target-specific attribute

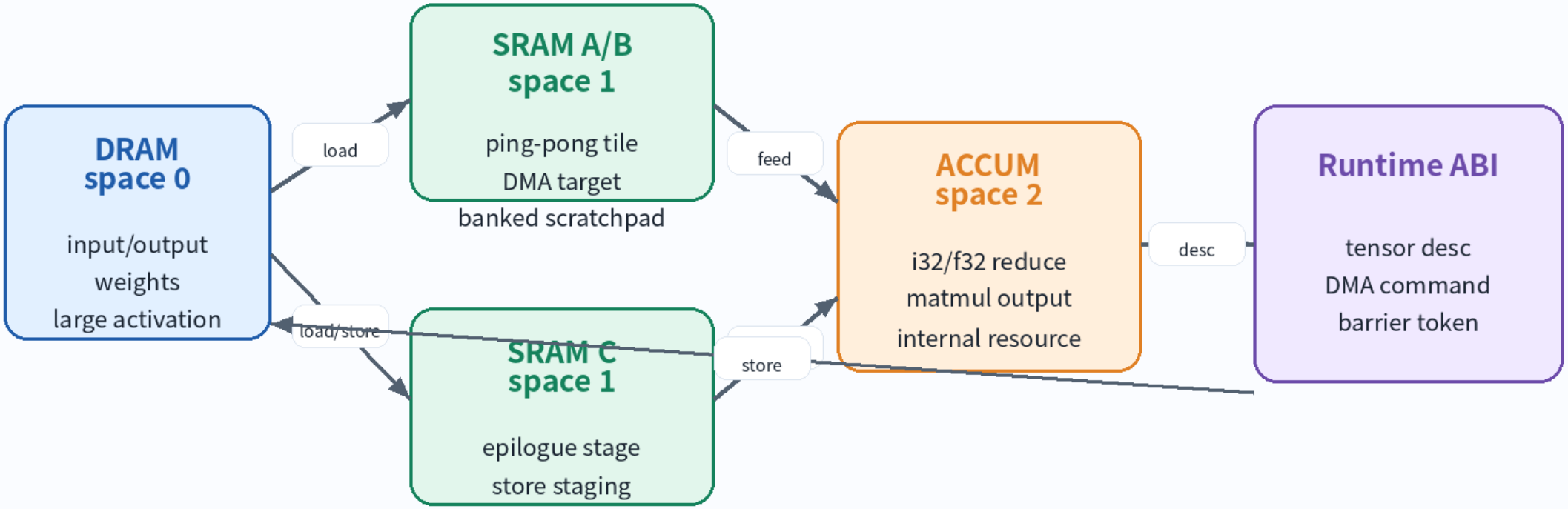
단순 숫자가 아니라 allocation/placement/ABI contract

DRAM/SRAM/ACCUM/CONST semantics 를 verifier 가 알아야 함

custom attribute 가 bank, align, scope 표현에 유리함

NPU Memory Space Mapping

DRAM / SRAM / ACCUM Mapping



Rule: memory_space is a placement + legality + descriptor contract, not only a numeric tag.

Memory Space Examples

```
// Course convention: 0=DRAM, 1=SRAM, 2=ACCUM
memref<1024x1024xf16, 0>     // DRAM tensor
memref<64x128xf16, 1>        // SRAM tile
memref<64x64xi32, 2>         // ACCUM tile

// Real backend style
memref<64x128xf16, strided<[160,1], offset:0>,
       #npu.mem<"sram", bank=0, align=64, scope="tile">>
```

DRAM Space Contract

host/runtime-visible tensor storage

DMA source/sink for activations and weights

alignment and address range must be legal

large tensors generally stay here until tiled

SRAM Space Contract

scratchpad tile buffer with explicit lifetime

bank assignment and double buffering are compiler decisions

layout padding may trade footprint for bandwidth

illegal escape to function boundary should be diagnosed

ACCUM Space Contract

matmul/reduction accumulator resource

usually wider dtype: i32/f32

may not be byte-addressable or host-visible

epilogue/requant must consume before store staging

Memory Space Cast vs DMA

memory_space_cast can alias same underlying memory with a different memref type

DMA copy moves data between spaces

Do not use cast as a substitute for DMA

bubble-down-memory-space-casts can simplify generic/default casts

MatMul Tile Mapping Pseudo IR

```
%a_tile = memref.alloc() {alignment = 64} : memref<64x128xf16, 1>
%b_tile = memref.alloc() {alignment = 64} : memref<128x64xf16, 1>
%c_acc  = memref.alloc() {alignment = 64} : memref<64x64xi32, 2>
%c_sram = memref.alloc() {alignment = 64} : memref<64x64xi8, 1>

// npu.dma_load A -> a_tile
// npu.dma_load B -> b_tile
// npu.matmul a_tile, b_tile -> c_acc
// npu.requant c_acc -> c_sram
// npu.dma_store c_sram -> C
```

Footprint Formula

A_tile = Mt * Kt * sizeof(A)

B_tile = Kt * Nt * sizeof(B)

C_acc = Mt * Nt * sizeof(accum)

SRAM_total = pingpong*(A+B) + C_stage + tags + padding

Tile Size Heuristics

SRAM fit is necessary but not sufficient

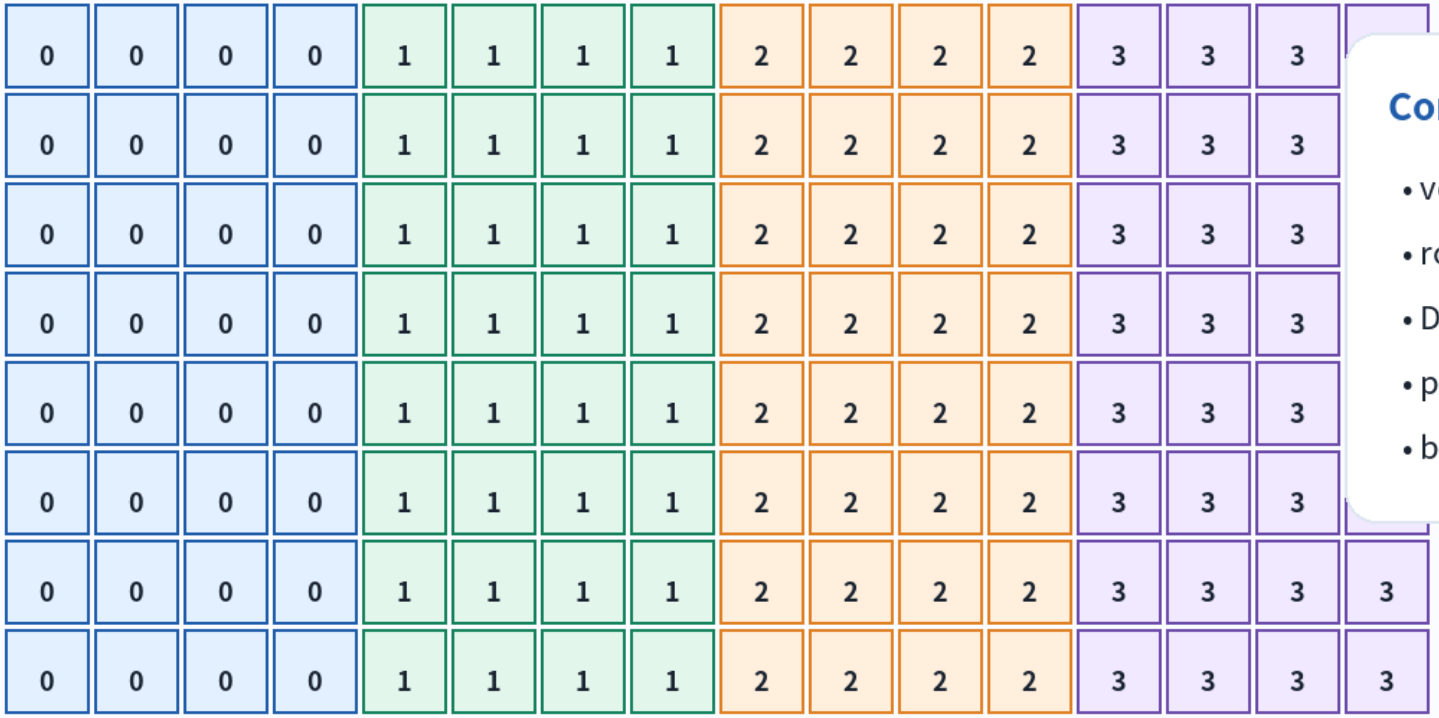
prefer Kt that maximizes data reuse

align rows to DMA burst and vector lane width

pad strides to reduce bank conflicts when needed

SRAM Banking and Padding

SRAM Bank / Padding Intuition



Compiler checks

- vector lane access pattern
- row stride alignment
- DMA burst size
- padding footprint
- bank assignment attr

Example: bank = floor(col / 4) mod 4. Padding row stride can shift the next row and reduce repeated same-bank pressure.

Runtime Descriptor

rank, sizes, strides, dtype, memory_space are descriptor fields

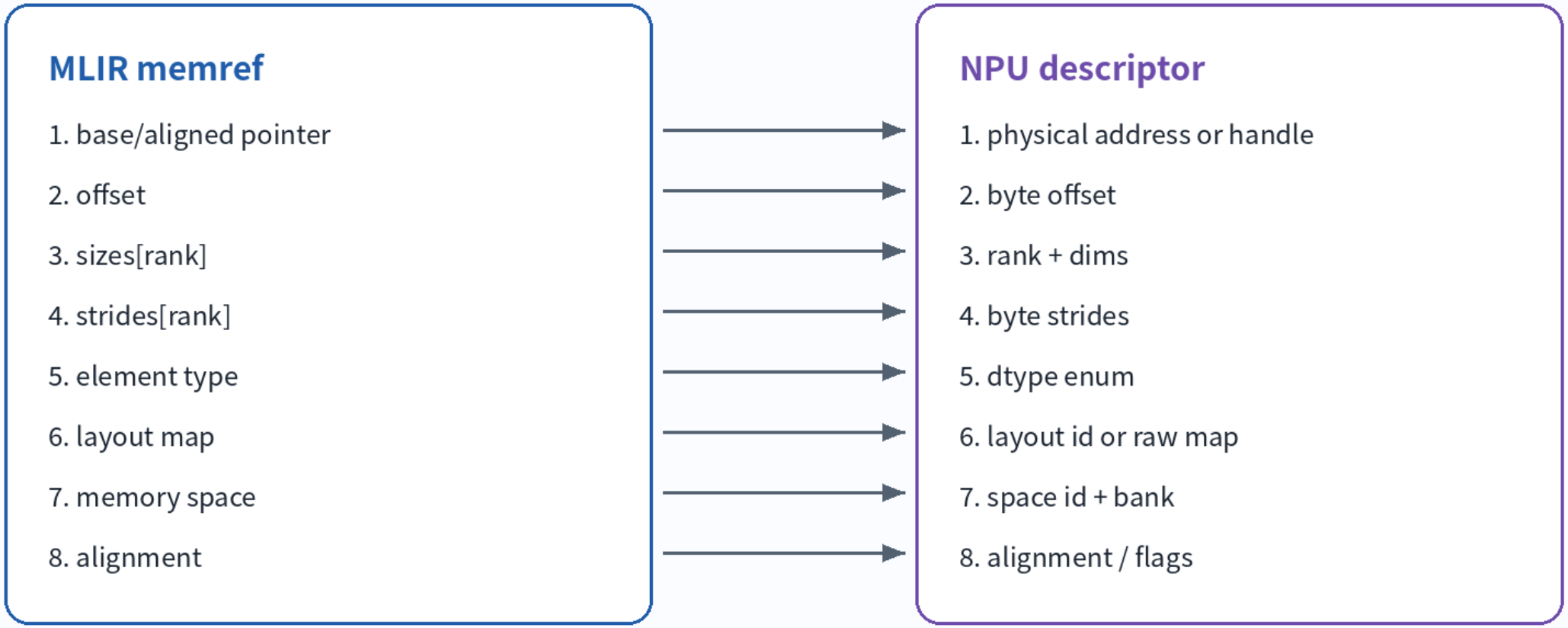
static-only kernels may bake constants into command opcode

dynamic shape kernels need runtime descriptor metadata

layout_id is compact but less flexible than raw strides

Runtime Descriptor Lowering

MemRef Metadata -> NPU Runtime Descriptor



Verifier: type/layout/space must match kernel ABI

Pass Design

Classify buffers by role: input/output/weight/tile/accum/tag

Analyze lifetime and capacity before rewriting types

Assign layout and bank together, not independently

Emit diagnostics for unsupported layout or illegal escape

Recommended Pass Position

one-shot-bufferize

npu-classify-buffers

npu-assign-memory-spaces

npu-propagate-layout

npu-insert-dma

npu-lower-runtime-descriptors

Verifier Checklist

shape/layout/type match kernel contract

SRAM allocations fit capacity and lifetime plan

ACCUM buffers do not escape illegally

DMA source/destination spaces and alignment are legal

dynamic offset/stride metadata is preserved

FileCheck Strategy

CHECK expected memory space on alloc/subview types

CHECK-NOT unexpected memref.copy

negative test for unsupported layout

test dynamic descriptor fields after lowering

test memory_space_cast cleanup when expected

Exercise A: MemRef Annotation

Annotate shape, dtype, layout, dynamic metadata, memory space

Write one verifier rule for each field

Exercise B: Subview Tile Descriptor

Create 64x128 tile view at M/K offset

Compute result offset, sizes, strides

Mark what becomes DMA descriptor field

Exercise C: MatMul Memory Map

Map A/B/C buffers to DRAM/SRAM/ACCUM

Calculate SRAM and ACCUM footprint

Choose bank and alignment policy

Write runtime descriptor draft

Common Mistakes

Treating memref as a raw pointer

Using memory_space_cast when DMA copy is required

Letting ACCUM buffers escape to host-visible ABI

Lowering layout too early into scalar address arithmetic

Ignoring dynamic strides in descriptor ABI

Takeaways

MemRef is the NPU storage contract after bufferization

Layout controls address generation and data movement efficiency

Memory space is target-specific and must be verified

DRAM/SRAM/ACCUM mapping is correctness + performance + ABI

Next: Vector Dialect and tensorization to NPU matmul

References

MLIR MemRef Dialect

MLIR Data Layout Modeling

MLIR Passes and memory-space cast cleanup

MLIR Builtin Dialect - MemRef type

MLIR FAQ - MemRef is not only a pointer